

**Profile ORM object****Profile ORM model**

```
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    city = models.CharField(max_length=50)
```

The field relates  
to the User object.

User-specific  
data

**UserProfile table**

| ID | user | city      |
|----|------|-----------|
| 21 | 1    | Detroit   |
| 22 | 2    | Liverpool |

**Django user ORM table**

| ID | username | password     | is_admin | ... |
|----|----------|--------------|----------|-----|
| 1  | admin    | GHLKJRjd...  | 1        |     |
| 2  | ringo    | \$RXJARKe... | 0        |     |

Many more  
fields

Encrypted  
string

**Replacing the user model with a custom model****Custom manager and custom user ORM model**

```
class LoginManager(BaseUserManager):
    def create_user(self, email, password=None, **kwargs):
        # Code for creating users

    def create_superuser(self, email, password, **kwargs):
        # Code for creating admins
```

```
class Login(AbstractBaseUser):
    email = models.EmailField(max_length=255, unique=True)
    is_active = models.BooleanField(default=True)
    is_admin = models.BooleanField(default=False)
    city = models.CharField(max_length=50)
```

User-specific  
data

Associated with a  
custom manager

Uses the email  
as login ID

**Custom user ORM table (login)**

| ID | password     | email             | is_active | is_admin | city      |
|----|--------------|-------------------|-----------|----------|-----------|
| 1  | GHLKJRjd...  | admin@example.com | 1         | 1        | Detroit   |
| 2  | \$RXJARKe... | ringo@example.com | 1         | 0        | Liverpool |

**Django user ORM table**

| ID | username | password | is_admin | ... |
|----|----------|----------|----------|-----|
|----|----------|----------|----------|-----|

Table left empty

Figure 6.2 User profiles vs. replacing the User ORM object for user data storage